

Programmierung

Vorlesung, Übung und Tutorium

07 – Sortiervverfahren

Prof. Dr. Andreas Biesdorf

Tutorium: Nicolas Pfeifer

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Agenda

1 Software Engineering vs.
Programmierung

2 Testen in Python

3 Argumente und
Kommandozeilenbefehle

4 Datei-Handling und IO

5 Algorithmen und Datenstrukturen

6 Such- und Sortierverfahren



Portfolioprüfung

- **Eingabe:**
Folge von Zahlen $\langle a_1, a_2, a_3, \dots, a_n \rangle$
- **Ausgabe:**
Sortierte Folge der Eingabe $\langle a'_1, a'_2, a'_3, \dots, a'_n \rangle$
mit $a'_1 \leq a'_2 \leq a'_3 \leq \dots \leq a'_n$
- **Aufgabe:**
Finde einen möglichst effizienten Weg zur Sortierung der Eingabemenge

„Naive“ Sortierverfahren

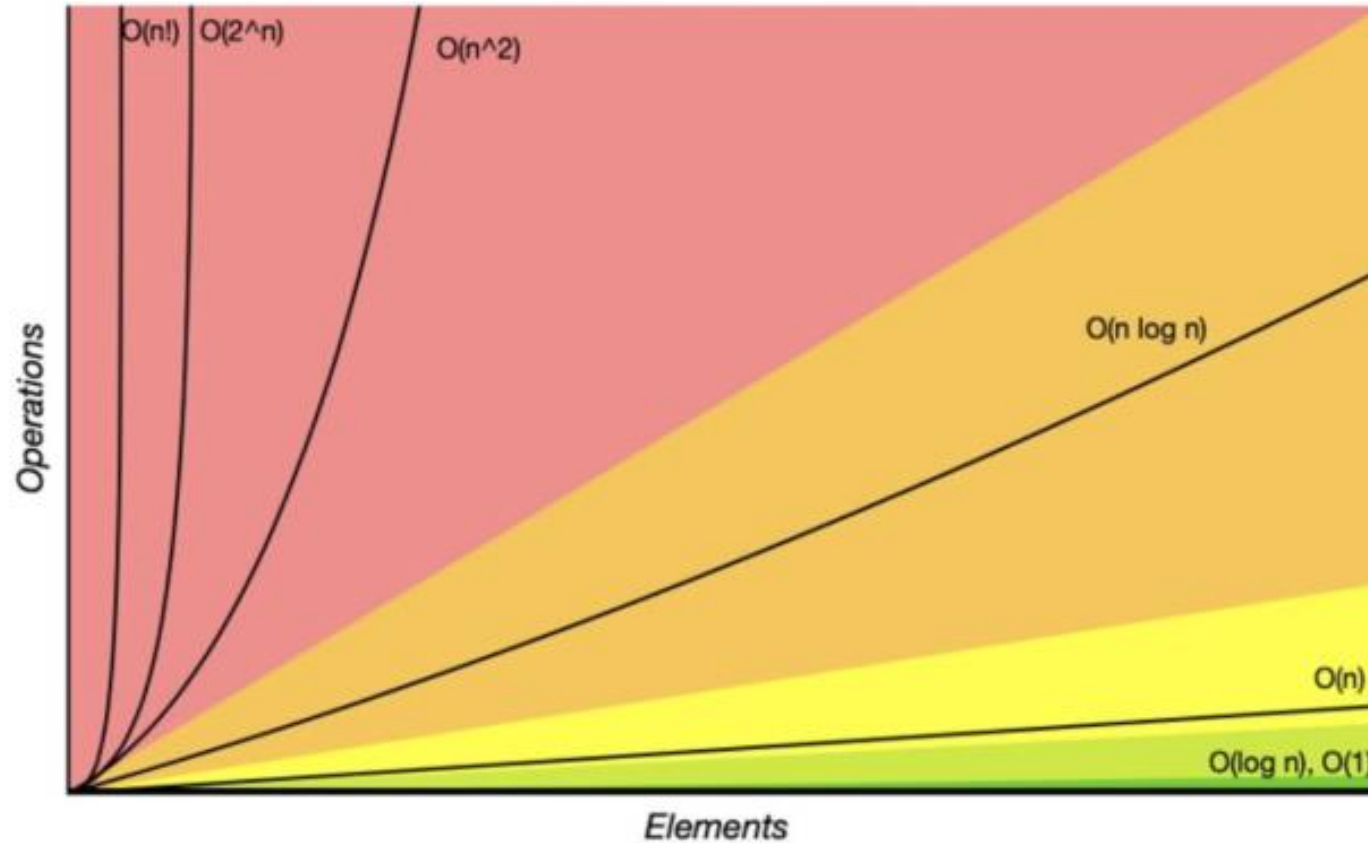
- Bubble-Sort
- Selection-Sort
- Insertion-Sort

Die drei Verfahren haben eine durchschnittliche Zeitkomplexität von $O(n^2)$

Divide-and-Conquer

- Quicksort
- Mergesort
- Heapsort

Mergesort und Quicksort gehören zu den effizientesten Sortieralgorithmen mit einer durchschnittlichen Zeitkomplexität von $O(n \log n)$

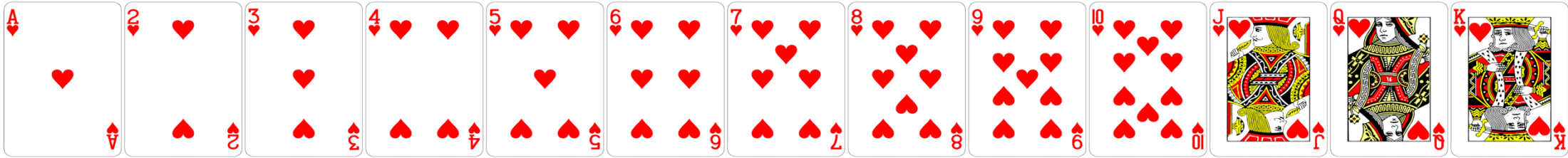


<https://builtin.com/software-engineering-perspectives/nlogn>

SORTIEREN VON SPIELKARTEN – DREI „INTUITIVE“ VERFAHREN

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R



Bubble Sort

- Aufnehmen aller Karten vom Tisch
- vertausche ggf. benachbarte Karten, bis Reihenfolge korrekt

Selection Sort

- Aufnehmen der jeweils niedrigsten Karte vom Tisch
- Anfügen der Karte am Ende

Insertion Sort

- Aufnehmen einer beliebigen Karte
- Einfügen der Karte an der korrekten Position

Bubble-Sort

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Idee: lasse die größten Zahlen in der Menge „aufsteigen“:

- Durchlaufe die Menge an Zahlen und vergleiche jeweils zwei nebeneinander liegende Zahlen
- Falls $a_n > a_{n+1}$: vertausche die beiden Zahlen
- Nach dem ersten Durchlauf ist die letzte Zahl garantiert die größte Zahl
- Wiederhole von Anfang an



BEISPIEL BUBBLE SORT

Erste Iteration:						
55	7	78	12	42		swap
7	55	78	12	42		
7	55	78	12	42		swap
7	55	12	78	42		swap
7	55	12	42	78		78 ist die größte Zahl in der Menge
Zweite Iteration						
7	55	12	42	78		
7	55	12	42	78		swap
7	12	55	42	78		swap
7	12	42	55	78		
7	12	42	55	78		55 ist die zweitgrößte Zahl in der Menge
Dritte Iteration:						
7	12	42	55	78		
7	12	42	55	78		
7	12	42	55	78		
7	12	42	55	78		42 ist die drittgrößte Zahl in der Menge

```
def bubble_sort(numbers):  
    n = len(numbers)  
    for i in range(n - 1):  
        list_sorted = True  
        for j in range(0, n - i - 1):  
            if numbers[j] > numbers[j + 1]:  
                # Tausche die Positionen  
                numbers[j], numbers[j + 1] = numbers[j + 1], numbers[j]  
            list_sorted = False  
        if list_sorted:  
            # kein Vertauschen erforderlich gewesen  
            return
```

Selection-Sort

Idee: Suche iterativ das kleinste Element und lege es an die vorderste Stelle

- durchlaufe die Menge, finde das kleinste Element
- vertausche das kleinste Element mit dem ersten Element
- vorderer Teil ist sortiert (k Elemente nach Durchlauf k), hinterer Teil ist unsortiert (n-k Elemente)
- durchlaufe die hintere, nichtsortierte Teilmenge, finde das n-kleinste Element
- vertausche das n-kleinste Element mit dem n-ten Element

BEISPIEL SELECTION SORT

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

55	7	78	12	42		tausche 7 mit 55
7	55	78	12	42		tausche 12 mit 55
7	12	78	55	42		tausche 42 mit 78
7	12	42	55	78		tausche 55 mit 55

```
def selection_sort(numbers):  
    n = len(numbers)  
    for i in range(n):  
        # Finde das Minimum im unsortierten Teil der Liste  
        min_index = i  
        for j in range(i + 1, n):  
            if numbers[j] < numbers[min_index]:  
                min_index = j  
  
        # Tausche das Minimum mit dem aktuellen Element  
        numbers[i], numbers[min_index] = numbers[min_index], numbers[i]
```

Insertion-Sort

Wirtschaft
Hauptcampus

H	O	C	H		
S	C	H	U	L	E
T	R	I	E	R	

Idee:

- Teile die Menge in einen sortierten und unsortierten Bereich
- Entnahme der unsortierten Menge und füge es an der korrekten Stelle im sortierten Bereich ein
- Wiederhole bis die unsortierte Menge leer ist

BEISPIEL INSERTION SORT

						sortiert	unsortiert	Pivot-E':	
55	7	78	12	42		[0]	[1..4]	7	füge 7 vor 55 ein
7	55	78	12	42		[0..1]	[2..4]	78	füge 78 nach 55 ein
7	55	78	12	42		[0..2]	[3..4]	12	füge 12 vor 55 ein
7	12	55	78	42		[0..3]	[4]	42	füge 42 vor 55 ein
7	12	42	55	78		[0..4]	[]		

CODE INSERTION-SORT

```
def insertion_sort(numbers):  
    n = len(numbers)  
    for i in range(1, n):  
        pivot = numbers[i]  
        j = i - 1  
        while j >= 0 and numbers[j] > pivot:  
            numbers[j + 1] = numbers[j]  
            j -= 1  
        numbers[j + 1] = pivot
```

Quicksort

Wirtschaft
Hauptcampus

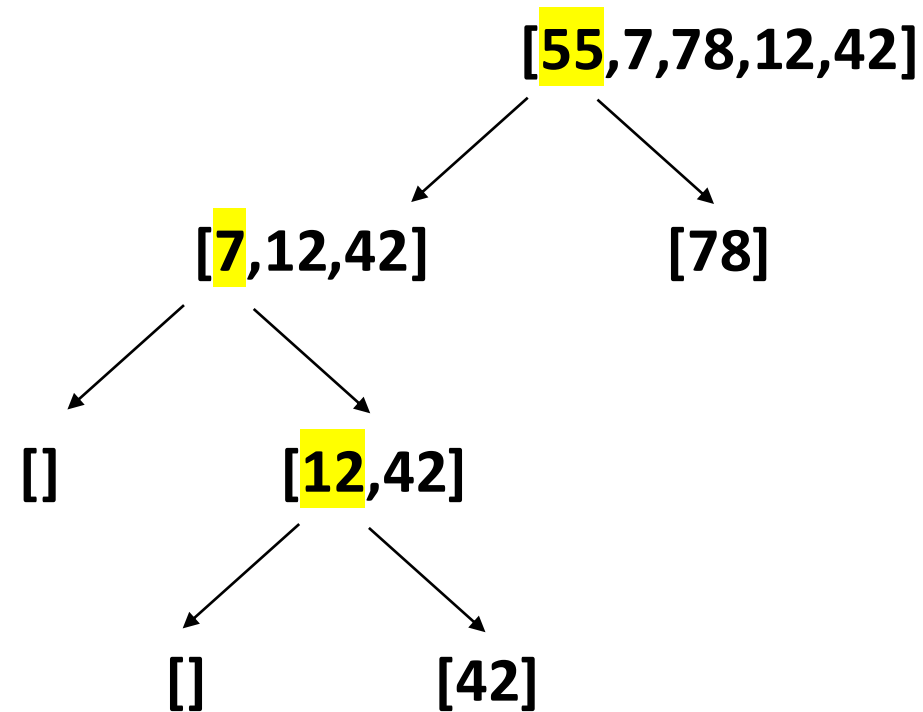
H O C H
S C H U L E
T R I E R

Idee: Teile-und-Herrsche

- **Pivot wählen:** Wähle ein Element aus der Liste als Pivot. Der Pivot ist das Element, um das herum die Liste partitioniert wird.
- **Partitionierung:** Durchlaufe die Liste und teile sie mit Hilfe des Pivot-Elements in zwei Teile
- **Rekursive Sortierung:** Wende den obigen Schritt (Pivot wählen und Partitionierung) rekursiv auf die beiden Teillisten an, die sich links und rechts des Pivots befinden. Dies wird so lange wiederholt, bis die Teillisten nur noch ein Element enthalten (was bereits sortiert ist).
- **Kombinieren:** Kombiniere die sortierten Teillisten, indem du die Teilliste mit den Elementen kleiner oder gleich dem Pivot, den Pivot selbst und die Teilliste mit den Elementen größer als der Pivot zusammenfügst.

Durch die rekursive Anwendung dieser Schritte wird die Liste in immer kleinere Teillisten zerlegt und anschließend durch die Kombination der sortierten Teillisten in der richtigen Reihenfolge wieder zusammengesetzt.

BEISPIEL QUICKSORT



```
def quicksort(numbers):  
    if len(numbers) <= 1:  
        return numbers  
    else:  
        pivot = numbers[0]  
        less = [x for x in numbers[1:] if x < pivot]  
        greater = [x for x in numbers[1:] if x > pivot]  
        return quicksort(less) + [pivot] + quicksort(greater)
```

Achtung: Vergleich lediglich auf < oder > eliminiert Duplikate. Sofern Duplikate erhalten bleiben sollen, muss einer der beiden Vergleiche auf <= verglichen

Mergesort

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Idee: Teile-und-Herrsche

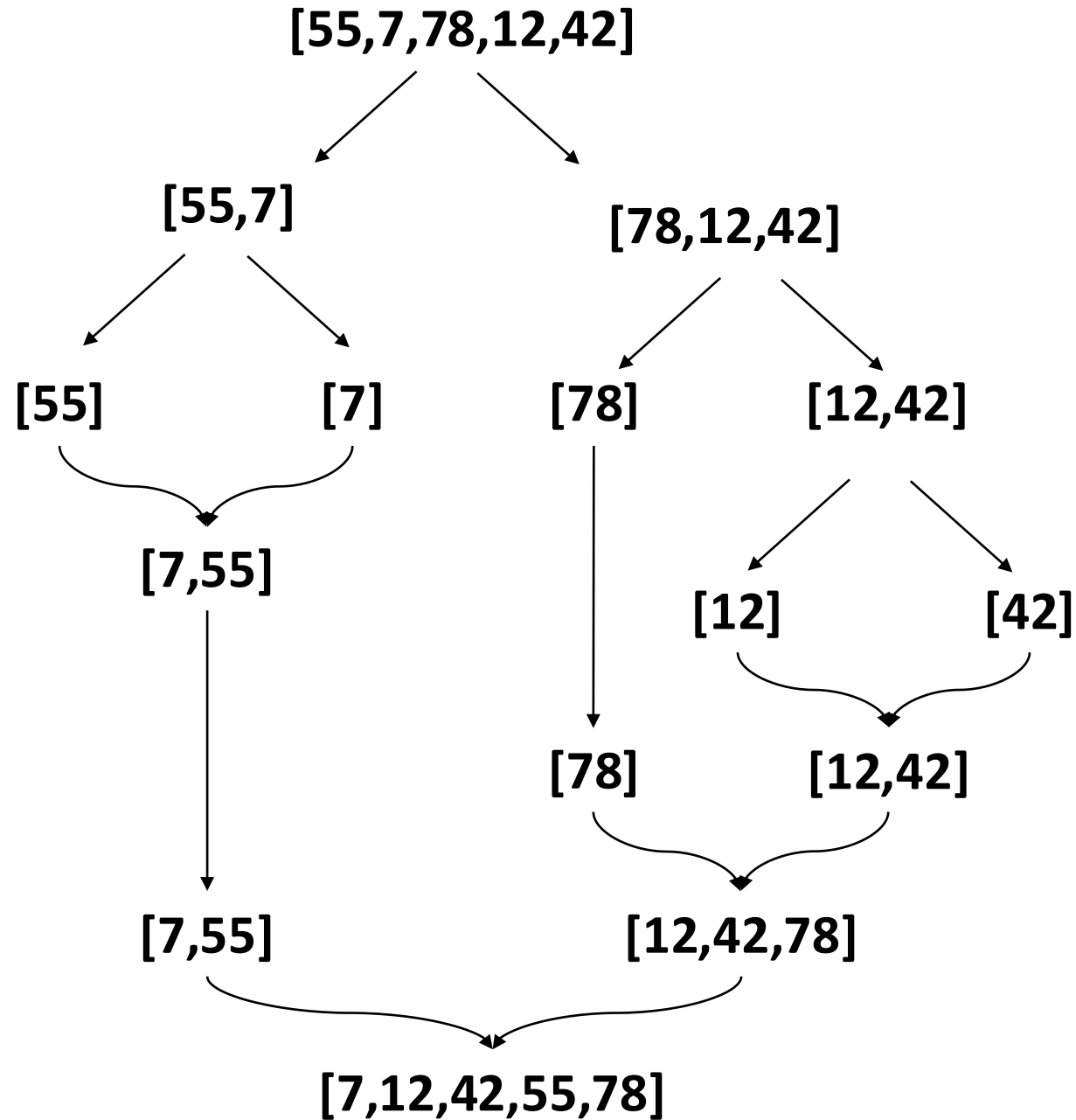
- **Teilen:** Teile die Liste rekursiv in immer kleinere Teillisten, bis jede Teilliste nur noch ein Element enthält. Dies wird erreicht, indem die Liste in der Mitte geteilt wird, um zwei etwa gleich große Teillisten zu erhalten. Dieser Schritt wird rekursiv angewendet, bis nur noch Teillisten mit einem Element übrig sind.
- **Sortieren:** Sortiere jede Teilliste einzeln. Für eine Teilliste mit nur einem Element gilt sie bereits als sortiert. Bei Teillisten mit mehr als einem Element wird Mergesort rekursiv aufgerufen, um diese Teillisten weiter in kleinere Teillisten aufzuteilen und zu sortieren.
- **Merge:** Führe die sortierten Teillisten schrittweise zusammen, um eine größere sortierte Liste zu erzeugen. Dies wird erreicht, indem die Elemente der Teillisten verglichen und in die richtige Reihenfolge gebracht werden. Dabei werden die kleinsten Elemente zuerst in die resultierende Liste eingefügt. Dieser Schritt wird wiederholt, bis alle Teillisten zusammengeführt sind und die gesamte Liste sortiert ist.

Durch die rekursive Anwendung dieser Schritte wird die Liste in immer kleinere Teillisten zerlegt und anschließend durch die Kombination der sortierten Teillisten in der richtigen Reihenfolge wieder zusammengesetzt.

BEISPIEL MERGESORT

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R



```
def merge_sort(numbers):  
    if len(numbers) <= 1:  
        return numbers  
  
    # Teile die Liste in zwei Hälften  
    mid = len(numbers) // 2  
    left_half = numbers[:mid]  
    right_half = numbers[mid:]  
  
    # Wende Mergesort rekursiv an  
    left_half = merge_sort(left_half)  
    right_half = merge_sort(right_half)  
  
    # Führe die sortierten Hälften zusammen  
    return merge(left_half, right_half)
```

```
def merge(left, right):  
    merged = []  
    left_index = 0  
    right_index = 0  
  
    # Füge die Elemente in der richtigen Reihenfolge zusammen  
    while left_index < len(left) and right_index < len(right):  
        if left[left_index] < right[right_index]:  
            merged.append(left[left_index])  
            left_index += 1  
        else:  
            merged.append(right[right_index])  
            right_index += 1  
  
    # Füge die restlichen Elemente hinzu  
    merged.extend(left[left_index:])  
    merged.extend(right[right_index:])  
  
    return merged
```

Heap-Sort

Wirtschaft
Hauptcampus

H O C H
S C H U L E
T R I E R

Repräsentation der Daten durch einen Max-Heap

- Binärbaum, in dem der Schlüssel eines Knoten immer größer ist als die Schlüssel der beiden Nachfolgeknoten
- Wurzelknoten enthält größtes Element

Iteratives Vorgehen

- Entnahme des Wurzelknotens als größtes Element
- Herstellen eines Max-Heaps aus den verbleibenden Elementen

Motivation

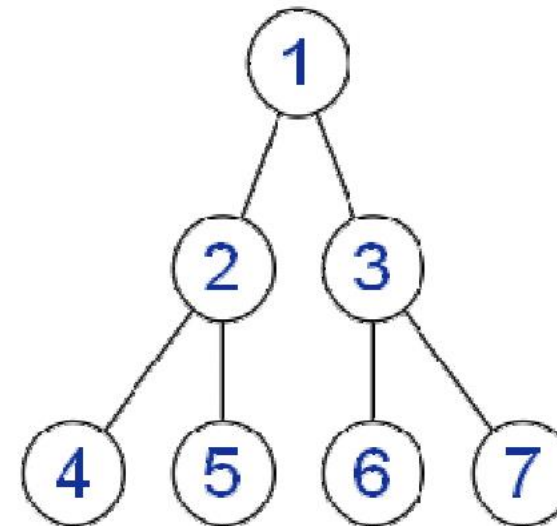
- n Iterationen mit Aufwand $\log(n)$
 - n Elemente müssen entnommen werden
 - Wiederherstellung des Heaps in $\log(n)$

Elemente eines Feldes können als Binärbaum angeordnet werden

Indizes eines Nachfolgeknotens k im Baum ergibt sich aus $2k$ und $2k+1$



Indizes eines Feldes



Repräsentation der
Indizes durch einen
Binärbaum

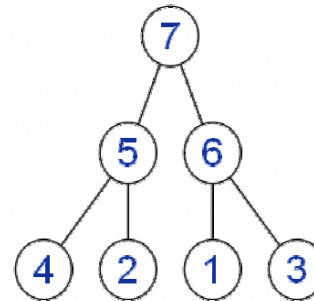
Folge von Daten ist ein Max-Heap, wenn

- die Schlüssel beider Nachfolgeknoten eines jeden Knotens k kleiner oder gleich sind als der Schlüssel von Knoten k
- der Schlüssel des Feldes an Stelle k größer oder gleich ist wie die Schlüssel an den Stellen $2k$ und $2k+1$
- über die Relationen anderer Knoten ist nichts bekannt
- aus den Anforderungen ergibt sich, dass der Wurzelknoten den größten Schlüssel enthält

Beispiel: Folge ist ein Max-Heap



Feldrepräsentation



Baumrepräsentation

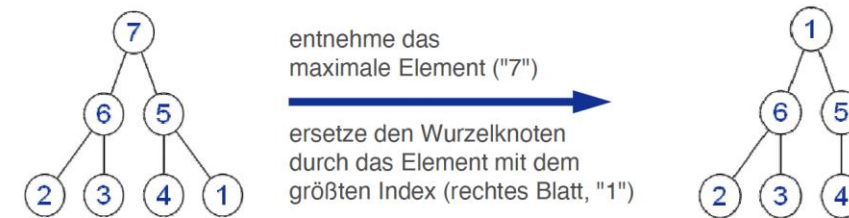
Initialisierung

- Aufbau eines Max-Heaps für ein Feld

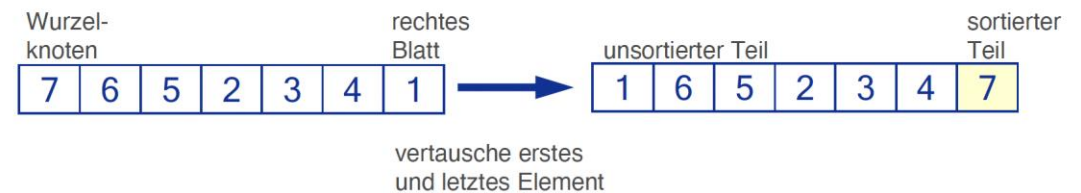
N Iterationen

- Entnahme des größten Elements (Wurzelknoten)
- Wiederherstellung des Max-Heaps aus den verbleibenden Elementen

■ Heap-Repräsentation



■ Feldrepräsentation



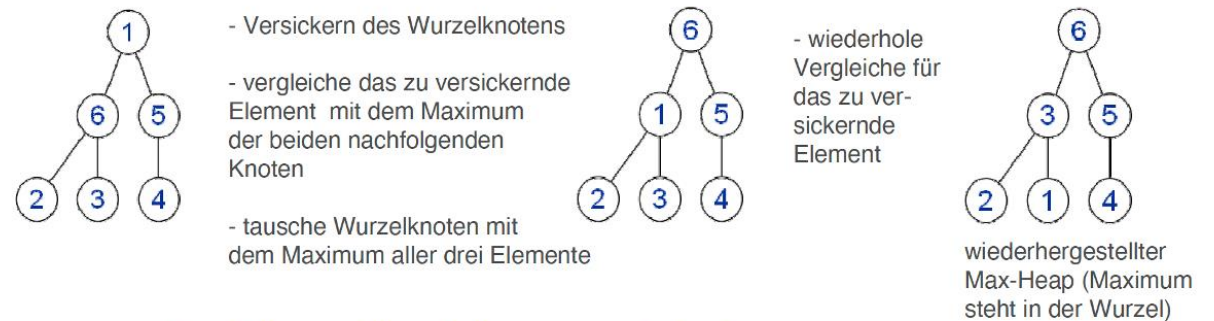
Initialisierung

- Aufbau eines Max-Heaps für ein Feld

N Iterationen

- Entnahme des größten Elements (Wurzelknoten)
- Wiederherstellung des Max-Heaps aus den verbleibenden Elementen

- Wiederherstellung eines Max-Heaps (versickern eines Elements, percolate)
- Heap-Repräsentation



- maximal $\log n$ Versickerungs-Schritte

Initialisierung

- Aufbau eines Max-Heaps für ein Feld

N Iterationen

- Entnahme des größten Elements (Wurzelknoten)
- Wiederherstellung des Max-Heaps aus den verbleibenden Elementen

- Wiederherstellung eines Max-Heaps (versickern eines Elements, percolate)

- Feldrepräsentation

	1	2	3	4	5	6	7
a	1	6	5	2	3	4	7

- Versickern des ersten Elements $a[1]$
- Vergleiche mit Maximum der Elemente $a[2 \cdot 1]$ und $a[2 \cdot 1 + 1]$
- Tausche entsprechend

	1	2	3	4	5	6	7
a	6	1	5	2	3	4	7

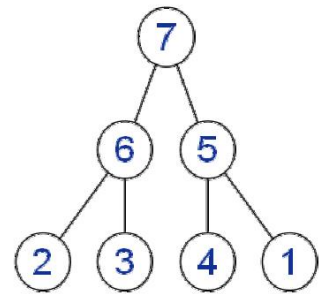
- Vergleiche $a[2]$ mit Maximum der Elemente $a[2 \cdot 2]$ und $a[2 \cdot 2 + 1]$
- Tausche entsprechend

	1	2	3	4	5	6	7
a	6	3	5	2	1	4	7

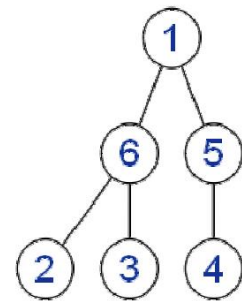
wiederhergestellter
Max-Heap (Maximum
von $a[1..6]$ steht in $a[1]$)
 $a[7]$ ist bereits sortiert

- maximal $\log n$ Versickerungs-Schritte

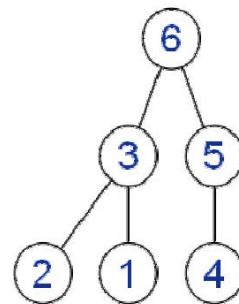
HEAP-SORT: BEISPIEL



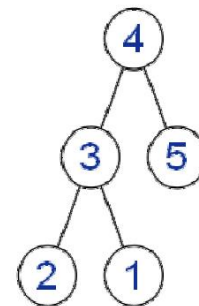
entnehme "7"



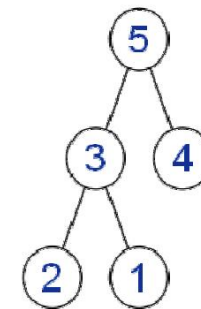
"1" neue Wurzel



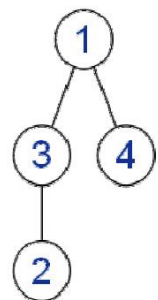
versickere "1"



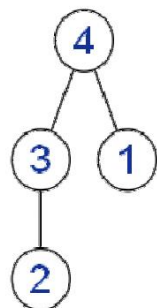
entnehme "6",
"4" neue Wurzel



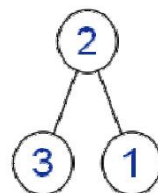
versickere "4"



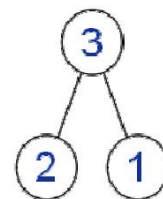
entnehme "5",
"1" neue Wurzel



versickere "1"



entnehme "4",
"2" neue Wurzel



versickere "2"



HEAP-SORT: BEISPIEL

7	6	5	2	3	4	1
1	6	5	2	3	4	7
6	3	5	2	1	4	7
4	3	5	2	1	6	7
5	3	4	2	1	6	7
1	3	4	2	5	6	7
4	3	1	2	5	6	7
2	3	1	4	5	6	7
3	2	1	4	5	6	7

entnehme "7", setze "1" an erste Position

versickere "1"

entnehme "6", setze "4" an erste Position

versickere "4"

entnehme "5", setze "1" an erste Position

versickere "1"

entnehme "4", setze "2" an erste Position

versickere "2"

...

CODE HEAP-SORT

```
def heap_sort(arr):  
    n = len(arr)  
  
    # Erstellen eines Max-Heap  
    for i in range(n // 2 - 1, -1, -1):  
        heapify(arr, n, i)  
  
    # Extrahieren der Elemente aus dem Heap  
    for i in range(n - 1, 0, -1):  
        arr[i], arr[0] = arr[0], arr[i]  
        heapify(arr, i, 0)
```

```
def heapify(arr, n, i):  
    largest = i  
    left = 2 * i + 1  
    right = 2 * i + 2  
  
    if left < n and arr[left] > arr[largest]:  
        largest = left  
  
    if right < n and arr[right] > arr[largest]:  
        largest = right  
  
    if largest != i:  
        arr[i], arr[largest] = arr[largest], arr[i]  
        heapify(arr, n, largest)
```

Verfahren	bester Fall	mittlerer Fall	schlechtester Fall
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$
Heap	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$
Quick	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n^2)$
Merge	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$
Counting	$O(n)$	$O(n)$	$O(n)$